

Spécification d'Isopérimètre : eg-chessboard

1. Principes Fondamentaux d'Architecture

Pour éviter les divergences de comportement entre le modèle *pull/immutabilité* de React et le modèle *push/Proxy* de Vue 3 :

- Source de Vérité Unique** : L'état du jeu réside exclusivement dans `BoardCore`. Les frameworks ne font que refléter cet état graphiquement.
- Mutations Interdites depuis la Vue** : Aucun wrapper (React ou Vue) ne doit modifier directement l'objet `state` ou ses sous-propriétés (ex: `state.promotionDialogState`). Toute modification doit passer par une méthode publique de `BoardCore`.
- Flux Unidirectionnel avec Événements** : Le cœur notifie les wrappers de tout changement d'état interne via un mécanisme d'écoute (`onStateChange`). Le wrapper React met alors à jour son `useState` (via le getter public `core.getState()`), et le wrapper Vue met à jour sa référence réactive.
- Pas de Fuite d'Abstraction** : L'accès aux propriétés privées par contournement de typage (ex: `coreRef.current['state']`) est strictement interdit. Toutes les lectures d'état se font via les getters publics (`getState()`, `getCurrentComment()`, `getHistoryViewerState()`, `isViewingHistory()`, etc.).

2. Contrat d'Interface Public (Props & Typage)

Les types d'entrée proviennent exclusivement de `@lichess-org/chessground` (`Config`), de `chess.js` (`Move`), ou de nos structures communes (`StockfishConfig`, `ChessDiagram`). Les types `Key` et `DrawShape` sont ré-exportés à la racine du paquet `eg-chessboard`.

Valeurs par défaut

- React** : Assignées via la déstructuration ES6 au niveau des arguments du composant.
- Vue 3** : Assignées via la macro `withDefaults` en utilisant des fonctions *factory* pour les objets afin de prévenir les mutations de références partagées.

3. Matrice d'Isopérimètre et de Correspondance

Élément / Prop	Type	Nom React	Nom / Événement Vue 3	Stratégie d'alignement & Validation
Configuration	Prop	<code>boardConfig</code>	<code>boardConfig</code>	Alignement strict des types via <code>Config</code> .
Couleur Joueur	Prop	<code>playerColor</code>	<code>playerColor</code>	Types littéraux : <code>'white' \ 'black' \ 'both'</code> .

Élément / Prop	Type	Nom React	Nom / Événement Vue 3	Stratégie d'alignement & Validation
Mode Libre	Prop	<code>freeMode</code>	<code>freeMode</code>	Synchronisé dynamiquement via <code>useEffect</code> (React) et <code>watch</code> (Vue). L'écouteur <code>change</code> de Chessground (configuré dans <code>events.change</code>) synchronise automatiquement le jeu (<code>syncGameFromBoard</code>) lors de tout drag-and-drop en mode libre.
Mode Solo	Prop	<code>soloMode</code>	<code>soloMode</code>	Utilisé pour les exercices d'apprentissage (déplacements consécutifs sans alternance de tour).
Conteneur Fit	Prop	<code>fitContainer</code>	<code>fitContainer</code>	Applique la classe CSS <code>.fit-container</code> au conteneur principal <code>.main-wrap</code> pour étendre l'échiquier à 100% de la hauteur/largeur du conteneur parent.
Config Stockfish	Prop	<code>stockfishConfig</code>	<code>stockfishConfig</code>	Synchronisation dynamique des options du moteur (incluant <code>workerUrl</code> et <code>wasmUrl</code>). Si absente ou inactive, aucun Web Worker n'est créé.
Diagramme	Prop	<code>diagram</code>	<code>diagram</code>	Initialisation et mise à jour dynamique de la FEN et des formes (flèches/cercles) via <code>setDiagram(diagram)</code> .
Création du Board	Event	<code>onBoardCreated</code>	<code>board-created</code>	Transmet l'instance de <code>BoardCore</code> dès l'initialisation.
Mouvement	Event	<code>onMove</code>	<code>move</code>	Transmet un POJO <code>Move</code> (chess.js). Sans objet d'événement natif. Émis uniquement après la mise à jour graphique complète de l'échiquier (permettant un <code>undoLastMove</code> immédiat sans désynchronisation visuelle).
Échec	Event	<code>onCheck</code>	<code>check</code>	Transmet la couleur en paramètre (<code>string</code>).
Échec & Mat	Event	<code>onCheckmate</code>	<code>checkmate</code>	Transmet la couleur en paramètre (<code>string</code>).

Élément / Prop	Type	Nom React	Nom / Événement Vue 3	Stratégie d'alignement & Validation
Pat (Stalemate)	Event	<code>onStalemate</code>	<code>stalemate</code>	Signature pure sans paramètre.
Nulle (Draw)	Event	<code>onDraw</code>	<code>draw</code>	Signature pure sans paramètre.
Promotion requis	Event	<code>onPromotion</code>	<code>promotion</code>	Transmet un POJO avec les détails requis pour la promotion.
Indication IA	Event	<code>onStockfishHint</code>	<code>stockfish-hint</code>	Transmet le meilleur coup calculé (<code>string</code>).
Clic sur case	Event	<code>onSquareClick</code>	<code>square-click</code>	Transmet la case cliquée en paramètre (<code>string</code>).

4. Gestion de la Réactivité & Résolution du Risque d'État

Le risque majeur de désynchronisation de `this.state` dans `BoardCore` sous React est résolu par les implémentations suivantes :

Cycle de vie d'une action (Exemple de la fermeture du dialogue de Promotion) :

1. L'utilisateur sélectionne une pièce dans le composant `PromotionDialog`.
2. Le wrapper de framework intercepte l'action et appelle **exclusivement** la méthode publique du cœur :
 - **React** : `coreRef.current?.closePromotionDialog();`
 - **Vue 3** : `core.closePromotionDialog();`
3. `BoardCore` met à jour son état interne et déclenche le callback global de changement d'état.
4. Les wrappers réagissent au changement d'état global (via `core.getState()`) pour mettre à jour l'UI.

Implémentation attendue dans le composant React (`Chessboard.tsx`) :

```
// Interdit : Modifier l'état localement ou accéder aux membres privés via des casts
// setState(prev => ({ ...prev, promotionDialogState: { isEnabled: false } }))
// const s = core['state'];

// Recommandé : Passer par l'API publique et les getters du Core
const coreState = core.getState();
setState({
  showThreats: coreState.showThreats,
  freeMode: coreState.freeMode,
  soloMode: coreState.soloMode,
  promotionDialogState: { ...coreState.promotionDialogState },
  historyViewerState: { ...coreState.historyViewerState },
  currentComment: coreState.currentComment,
});
```

5. Gestion des annotations graphiques et commentaires PGN

Pour assurer l'uniformité du traitement des exercices et des PGN :

1. **Extraction automatique** : Le traitement des commentaires textuels et des balises propriétaires (`[%cal]` pour les flèches, `[%cp1]` pour les ronds) est opéré exclusivement par `BoardCore` dans `updateCommentAndShapes()`. Les wrappers ne doivent pas faire de parsing de commentaires PGN de leur côté.
2. **Champ d'état commun** : Le texte de commentaire nettoyé est exposé dans l'état commun sous la clé `currentComment`, accessible via le getter public `core.getCurrentComment()`.
3. **Méthodes de dessin et d'extraction publiques** : Toute opération de dessin dynamique (ex: `drawMove`, `drawCircle`, `setShapes`) ou d'extraction des formes posées sur l'échiquier (`getShapes()` : `DrawShape[]`) doit être invoquée via les méthodes publiques de `BoardCore`. Les paramètres de cases (`from`, `to`, `square`) acceptent de manière permissive le type `Key | string`.
4. **Enrichissement / Saisie de commentaires** : L'écriture de commentaires et d'annotations graphiques dans le PGN s'effectue uniquement via `core.setComment(text, shapes)` (cible le coup visualisé à l'écran) ou `core.setCommentAtPly(ply, text, shapes)`. Le PGN résultant est récupéré via `core.getPgn()`.
5. **Tracé interactif en mode historique** : Afin de permettre l'enrichissement graphique du PGN lors de la navigation dans l'historique, le plateau de jeu reste interactif pour le tracé (`viewOnly: false` configuré au niveau de Chessground). Pour empêcher toute modification de la position par l'utilisateur tout en autorisant le dessin de cercles et de flèches avec le clic droit/glisser, les déplacements de pièces sont totalement désactivés via le paramétrage de Chessground (`movable.color: undefined`, `movable.dests: undefined`, `movable.free: false`). Les modifications graphiques opérées par l'utilisateur déclenchent `drawable.onChange` et mettent automatiquement à jour le PGN via `setCommentAtPly` (sans redessiner les formes déjà tracées grâce au paramètre `updateBoardShapes = false`).

6. Gestion des restrictions de déplacements et aides pour les exercices

Pour restreindre dynamiquement les mouvements de l'utilisateur ou valider ses actions dans le cadre d'exercices d'apprentissage :

1. **Méthodes publiques exclusives** : Toute restriction ou vérification de coups doit s'appuyer sur les méthodes publiques de `BoardCore` :
 - `core.setCustomDests(dests: Map<Key, Key[]> | null)` : Définit explicitement les pièces et leurs cases de destinations autorisées.
 - `core.restrictMovesToPieces(squares: Key[] | null)` : Filtre automatiquement les coups légaux de la position pour n'autoriser le déplacement que des pièces situées sur les cases spécifiées.
 - `core.isSquareAttacked(square: Key, byColor: 'white' | 'black') : boolean` : Permet à l'application hôte de détecter si le joueur s'est déplacé sur une case menacée afin de déclencher un avertissement ou une réinitialisation de l'exercice.
 - `core.getPieces(): Map<Key, { type: string; color: 'w' | 'b' }>` : Retourne la liste des pièces restantes sur l'échiquier pour valider la capture complète des pièces adverses.
 - `core.getSoloHistory(): Move[]` : Retourne l'historique des coups joués en mode solo.

2. **Cycle de vie** : Les restrictions modifient le comportement interne de Chessground de manière persistante jusqu'à ce qu'elles soient réinitialisées en passant `null`. En `soloMode`, après chaque coup, le trait est automatiquement conservé sur la couleur de la pièce jouée pour autoriser les déplacements consécutifs.

7. Méthodes Utilitaires d'État & Lifecycle

Pour interroger l'état interne de l'échiquier et gérer le nettoyage de manière uniforme :

- `core.setPlayerColor(color: 'white' | 'black' | 'both'): void` : Définit dynamiquement la couleur du joueur autorisée aux déplacements sans qu'elle ne soit écrasée par le trait courant.
- `core.setConfig(config: Config, fillDefaults?: boolean): void` : Applique les modifications de configuration sur Chessground sans réinitialiser la FEN ni effacer la sélection si la FEN reste inchangée.
- `core.getOrientation(): 'white' | 'black'` : Retourne l'orientation actuelle du plateau de jeu.
- `core.getPlacementFen(): string` : Retourne la FEN de placement actuelle des pièces (sans les autres métadonnées de FEN) extraite directement de Chessground, sans dépendre de la validation de `chess.js` (utile en mode libre ou pour les FENs en cours d'édition).
- `core.getCurrentComment(): string` : Retourne le commentaire textuel du coup courant.
- `core.getHistoryViewerState(): Readonly<HistoryViewerState>` : Retourne l'état de visualisation de l'historique.
- `core.isViewingHistory(): boolean` : Indique si la vue navigue actuellement dans l'historique.
- `core.getInCheckColor(): 'white' | 'black' | null` : Retourne la couleur du joueur en échec (ou `null`).
- `core.getGameOverReason(lang?: 'fr' | 'en'): string` : Retourne la raison formatée de fin de partie (ex: "Échec et mat ! Les Blancs ont gagné.").
- `core.destroy(): void` : Libère proprement toutes les sous-ressources (Workers Stockfish, instance DOM Chessground). Appelé automatiquement au démontage des wrappers React (`useEffect` cleanup) et Vue 3 (`onUnmounted`).

8. Gestion des Diagrammes (FEN + Formes) et tolérance aux FENs invalides/incomplètes

Pour instancier et lire des diagrammes combinant une position FEN et des formes dessinées (flèches et cercles) :

1. **Méthode publique exclusive** : L'écriture et le chargement d'un diagramme s'effectuent via `core.setDiagram(diagram: ChessDiagram)`.
2. **Récupération de l'état** : L'état actuel est obtenu via `core.getDiagram()`, retournant un objet `{ fen: string, shapes: DrawShape[] }` où les formes proviennent directement du plateau de jeu (`this.board.state.drawables.shapes`).
3. **Synchronisation réactive** :
 - **React** : Le composant surveille la prop `diagram` via `useEffect` et appelle `coreRef.current?.setDiagram(diagram)`.
 - **Vue 3** : Le composant surveille la prop `diagram` via un `watch` avec `deep: true` et appelle `core.value?.setDiagram(diagram)`.

4. **Tolérance aux positions incomplètes/invalides** : Si la FEN chargée via `setPosition` ou `setDiagram` ne respecte pas les contraintes strictes de `chess.js` (par exemple absence de rois, échiquier vide ou pièce unique), `BoardCore` charge une position minimale contenant le trait (turn) approprié, retire les rois factices, puis injecte manuellement chaque pièce sur le plateau. Ceci prévient tout crash ou désynchronisation de l'état logique par rapport à l'affichage visuel de Chessground.